SFN-World experiment draft v0.4 · not yet learned vs. unlearnable

Nov 2, 2026 · @Julian

Overview

A fast synthetic test of target-path and SIGReg-budget interventions for SIGReg world models (LeWM-style). Draft v0.4, 2 Nov 2026. Supersedes v0.3 (v0.1–v0.3 kept as separate files).

Changes from v0.3.

1. SIGReg is no longer held fixed across arms. The design now has two knobs: the target-path gate G (as before) and a per-slice SIGReg weighting, factor B. The context path stays identical in every arm. Attribution rules are in §2.
2. New target-path arm M1d: M1b's lagged-predictor learning progress, measured in a slowly refreshed eigenbasis of the residual covariance instead of fresh random bases.
3. New weak predictor (linear, action-additive) and a per-direction shortcut score s. Used only by factor B, never by a target gate.
4. Factor B levels: B0 (LeWM SIGReg), B1 (SIGReg relaxed on shortcut directions), B2 (B1 plus a small shrink penalty), B-rand (B1 on a random subspace, control).
5. Stage 0b also fits the weak predictor on ground truth, giving a reference R^2_{lin} per group. §3 corrects the v0.3 note on linear predictability: for the rotating groups a 2×2 linear map reaches $|E\, e^{i\omega}|^2$, well above ρ^2.
6. New metrics: effective dims per group, shortcut-flag precision and recall, eigensubspace stability.
7. Budget is now 108 encoder runs plus 9 predictor-only runs, plus up to 15 conditional runs (D = 24 cell, factor B in Stage 2).

Carried over from v0.3: 8-group environment, capacity knob D, Stage 0b learning-time measurement, A0 vs A2 reading table, M1a/M1b/M1c.

Question. In a SIGReg + prediction-loss world model, features that are predictable but expensive to learn are removed from the latent before the predictor learns them. Can we separate, at the level of the loss, error that is epistemic (learnable, not yet learned: keep)

from error that is aleatoric (irreducible: may drop), and does any intervention keep the first kind while still discarding the second? New in v0.4: when the SIGReg budget is tight, does freeing it from cheaply predictable directions help keep the hard ones?

Cost. Vector observations, MLPs, infinite on-the-fly data; about 108 encoder runs of 1–2 GPU-minutes each plus 9 predictor-only runs of seconds each (estimate, verify on your hardware).

1. Literature check: has this been done?

No paper found that uses learning progress, ensemble disagreement or a heteroscedastic head to gate the encoder's target-path gradient in a JEPA, and no ablation combining SIGReg with an EMA target to measure retention of hard-to-predict features. The diagnosis itself is well supported. Treat "not found" as "not found by me", not "does not exist". The new v0.4 row (last) was not re-searched; its entries are from memory and need checking.

Idea	Closest prior work	What differs from the proposal
Diagnosis: JEPA keeps slow / easy features, drops fast ones	Sobal et al. 2022 (JEPA focuses on slow features; fails with fixed distractor noise). Klindt, LeCun, Balestriero 2026 (with anisotropic transition rates the encoder recovers the 2nd Hermite polynomial of the slow latent instead of the fast latent). Qiao et al. 2026 (AVL-JEPA: "causal dynamics information collapse" in LeWM).	Explain or name the failure. AVL fixes it with an auxiliary action-prediction loss, not a loss-level gating rule.

Idea	Closest prior work	What differs from the proposal
Stop-gradient / EMA target with SIGReg	LeWM and LeJEPA deliberately drop EMA and stop-grad. BYOL, SimSiam, I-JEPA use them against collapse. Littwin et al. 2024: even with stop-grad, deep linear JEPAs suppress low-regression-coefficient directions, more so with depth.	No retention ablation of SIGReg + EMA found. Littwin predicts stop-grad alone may only partially help.
LP gate	RHO-LOSS (Mindermann et al. 2022): samples that are learnable and not yet learnt. Schmidhuber compression progress, Oudeyer learning progress.	Sample selection or intrinsic reward, not per-direction gating of an encoder's target gradient.
Ensemble gate	Plan2Explore, PETS, DEUP; 2026 JEPA papers use disagreement as a diagnostic or planning bonus.	Exploration, planning, OOD detection; not gating representation learning.
Heteroscedastic gate	Var-JEPA (2026), EpicWorldModel (stochastic JEPA predictor).	Uncertainty for UQ, planning or anti-collapse, not deciding which features the encoder may compress.
Weak-model shortcut detection, budget reallocation (new)	Learning from Failure (Nam et al. 2020) and Just Train Twice (Liu et al. 2021): a deliberately biased or weak model flags easy examples, which are then down-weighted.	Sample reweighting in supervised learning. Here a weak predictor flags latent directions, and SIGReg's per-slice weights are lowered on them.

2. Two knobs: the gated target path and the SIGReg budget

Every arm is a pair (G, B): a gate G on the target path and a SIGReg weighting B. The context path is identical in all arms.

```
L = \lVert P(z_t, a_t) - \tilde z_{t+1} \rVert^2 + \lambda \,
\mathrm{SIGReg}_{\omega}(z) + \mu \, R_{\mathrm{shrink}}(z)
```

The encoder receives gradient through three routes:

- **Context path:** make z_t informative. Information-preserving. Identical in all arms.
- **Target path:** make z_{t+1} easier to hit. Information-destroying. Mapped through G.
- **SIGReg:** push the latent toward an isotropic Gaussian. It fixes the variance budget but not which information fills it. Weighted per slice by ω(w), set by factor B.

Target path. As in v0.3, the gate may be a full D×D matrix:

```
z_tilde = sg(z_next) + (z_next - sg(z_next)) @ G   # forward value unchanged,
                                                   # target-path gradient
mapped through G
```

G = I is LeWM. G = 0 is stop-grad. EMA is G = 0 plus a lagged target encoder. Per-dimension gates are the special case G = diag(g).

SIGReg budget. SIGReg averages an Epps–Pulley statistic over random unit directions w. Factor B weights each slice by $\omega(w) = \max(\omega_{min}, 1 - \Sigma_k s_k (w \cdot v_k)^2)$, with v_k the shortcut basis and $s_k \in [0, 1]$ the shortcut score (§6). B0 is ω ≡ 1 and μ = 0, which is LeWM. Weights are not renormalized, so B1 also lowers total SIGReg strength; B-rand applies the same loss of strength to a random subspace.

Attribution rules.

- Arms with the same B differ only in the target path. All Stage 1 target arms run at B0, so every v0.3 comparison is unchanged.
- Arms with the same G differ only in SIGReg weighting and the shrink term, so differences are attributable to the budget.
- B1 vs B-rand at the same G separates "which directions were relaxed" from "SIGReg got weaker". The Stage 0 λ sweep covers globally weaker SIGReg.

3. Environment: SFN-World v2 (8 groups)

Unchanged from v0.3 except a new reference column R^2_lin and a corrected note on linear predictability.

Why v0.1 was not enough. v0.1′s fast group was simultaneously fast, fully predictable, hard to learn and action-dependent, so a method that kept it could not tell us which property it responded to. Its noise group was pure i.i.d. noise, which every gate identifies trivially.

Three axes, separated. Timescale = lag-1 autocorrelation ρ. True predictability = R^2max, the fraction of next-step variance explainable by the best predictor given (x_t, a_t). Learning time = how long a predictor needs to reach R^2max.

#	Group (2 dims each)	Dynamics	ρ	R^2max	R^2_lin	Learn time
1	Slow OU	$s' = 0.98\,s + \sqrt{(1-0.98^2)}\,\varepsilon$	0.98	0.96	0.96	easy
2	Medium OU	$s' = 0.7\,s + \sqrt{(1-0.7^2)}\,\varepsilon$	0.7	0.49	0.49	easy
3	Slow + random jumps	$s' = s$; with prob. 0.05 resample $s \sim N(0, I)$	0.95	0.90 avg; 0 at jumps	0.90	easy
4	Fast, easy	$v = (\cos\varphi, \sin\varphi)$, $\varphi' = \varphi + \pi/2$ (no action)	0	1.0	1.0	easy (linear)
5	Fast, hard	$\theta' = \theta + \omega(a)$ [+ difficulty term], emb. $(\cos\theta, \sin\theta)$	≈ −0.15	1.0	0.41 (D1: 0.35)	hard
6	Fast, hard, partial	as 5, plus angle noise $\sigma = \sqrt{\ln 2} \approx 0.83$ rad	≈ −0.10	0.5	0.21	hard
7	Medium, hard	$\theta' = \theta + 0.3\cdot\omega(a)$ [+ D1 term]	≈ 0.83	1.0	0.93 (D1: 0.79)	medium–hard
8	Noise	i.i.d. $N(0, I)$ each step	0	0	0	—

Notes: $\omega(a) = (0.6, 1.4, 2.2, 3.0)$ rad for $a \in \{0..3\}$, a uniform random; groups 5–7 use independent angles and the same action. For group 6, R^2max = e^(−σ²) = 0.5. ρ values are approximate and are measured in Stage 0.

R^2_lin (new). Best linear predictor of the next step from the true current values plus an additive one-hot action, no action interaction. Computed numerically ($4\cdot10^5$ samples, θ uniform); D1 values approximate because D1 makes θ non-uniform. This is the ground-truth reference for the weak predictor (§6) and is re-measured in Stage 0b.

Correction to v0.3. v0.3 said a linear predictor already achieves ρ^2. That holds for the scalar OU groups but understates the rotating groups: a 2×2 linear map reaches $|E\, e^{i\omega}|^2$, so group 5 is 0.41 linearly predictable although $\rho^2 \approx 0.02$, and group 7 is 0.93. Consequence for factor B: group 7 at D0 will be flagged as a shortcut. Consequence for the 4 vs 5 test: group 5 at D0 is exactly solvable by a per-action linear map ($R^2 = 1.0$ with action interaction), so its hardness may be modest for an MLP. Stage 0b decides.

The learn-time column is an assumption until measured. Stage 0b measures it by training the predictor on the ground-truth state with no encoder (§8). If group 5 is not clearly slower to learn than group 4, the 4 vs 5 comparison does not test what it claims, and the environment must change first.

Ideal outcome. Keep every group with $R^2_{max} > 0$ (groups 1–7), drop only group 8. Group 3 should be kept including its post-jump values.

What each comparison isolates:

- 4 vs 5: same timescale, same predictability, different learning time. The direct test of "removed before the predictor learns it". Most important comparison.
- 5 vs 6 vs 8: same timescale, aleatoric fraction 0 / 0.5 / 1. Is a gate calibrated, or does it only detect pure noise?
- 5 vs 7: similar learning problem, different timescale.
- 1, 2, 8: classic slow-feature ordering, sanity check.
- 3: are rare unpredictable events thrown away together with noise?

Observation. $x = g([w_1 \cdot y_1, \ldots, w_8 \cdot y_8]) \in \mathbb{R}^{64}$, with y_k the 2-dim group values and g a frozen random 3-layer MLP (64 units, LeakyReLU, orthogonal init). Default salience $w = 1$; Stage 0 sweeps $w_{fast} \in \{0.3, 1\}$ jointly on groups 4–7.

Difficulty knob for groups 5–7 (Stage 2). D0 as in the table; D1 adds $0.8 \cdot \sin(2\theta)$ to the update; D2 replaces the update of groups 5–6 with the doubling map $\theta' = 2\theta + \omega(a)$ (group 7 stays at D1).

Capacity knob. Groups 1–7 hold 14 true dims. D = 16 is tight (2 spare dims), D = 48 is roomy. New: D = 24 is a conditional middle cell for factor B (Stage 1c), because at D = 16 there is almost no budget to reallocate. D = 8 stays rejected: it cannot hold the 14 dims that should be kept.

4. Model and training defaults (shared by all arms)

Component	Setting
Encoder	MLP 64→256→256→D, GELU, final BatchNorm1d (as in LeWM's projector). D = 16 default; 48 in Stage 1b; 24 in Stage 1c
Predictor	MLP (D + one-hot action)→256→256→D, GELU. One-step prediction, teacher forcing
SIGReg	Epps–Pulley, 256 random directions resampled each step, 17 knots; applied to z_t and z_{t+1}; λ = 0.1 (baseline λ swept in Stage 0 only). Per-slice weights ω(w) = 1 except under factor B1/B2/B-rand
Lagged predictor (M1 arms)	EMA copy P̄ of the predictor, τ_p = 0.99
Weak predictor (new; factor B only)	Linear (D + one-hot action)→D, no hidden layer, no action interaction. Trained on sg(z_t), sg(z_{t+1}) with its own MSE; never sends gradient to the encoder
Eigenbases (new)	U: eigenbasis of the EMA residual covariance of the full predictor (M1d). V: eigenbasis of the EMA covariance of the weak predictor's output (factor B). Both EMA τ = 0.99, refreshed by eigh every 100 steps
Factor B constants (new)	ω_min = 0.2, μ = 0.01 (B2 only), τ_w and T calibrated once in Stage 0
Optimizer	AdamW, lr 1e-3, wd 0, batch 512, 20k steps, cosine decay in the last 50%
Seeds	3 per cell (env mixing g fixed per seed index, shared across arms)

5. Arms

An arm is a target-path choice combined with a budget level, written as e.g. M1d+B1. Without a suffix an arm runs at B0.

Target-path arms (knob G).

ID	Arm	Target-path gate G	Extra cost
A0	LeWM baseline	G = I	—
A1	Stop-grad (α sweep)	G = α I, α ∈ {0, 0.1, 0.3}	—
A2	EMA target	G = 0; target = EMA encoder, τ ∈ {0.99, 0.996}	1 encoder copy
M1a	LP, per dimension	Lagged-predictor progress per latent dim, G = diag(g)	1 predictor copy
M1b	LP, random projections	Lagged-predictor progress per random direction, basis-free	1 predictor copy
M1c	LP, naive loss slope	Slope of running loss per fixed random direction (confound check)	—
M1d (new)	LP, residual eigenbasis	Lagged-predictor progress per direction of U, G = U diag(g) U^T	1 predictor copy, one D×D eigh per 100 steps
M2	Ensemble gate	Low gate where K = 5 heads disagree relative to their error	5 small heads
M3	Heteroscedastic gate	Gate from predicted vs observed residual variance	variance head
C1	Two-timescale (control)	G = I, predictor lr ×5	—
C2	Warm-up only (control)	G = 0 for the first 2k steps, then I	—
C3	Oracle (upper bound)	G = I + supervised aux loss z → groups 1–7	—

C1 and C2 matter: if either matches the gated arms, the cheap explanation ("the predictor just needs a head start") wins and the gating machinery is unnecessary.

Budget levels (knob B, new).

ID	SIGReg weighting	Shrink term	Extra cost
B0	$\omega \equiv 1$ (LeWM)	none	—
B1	$\omega(w) = \max(\omega_min, 1 - \Sigma_k s_k (w \cdot v_k)^2)$	none	weak predictor, one D×D eigh per 100 steps
B2	as B1	$\mu \Sigma_k s_k \mathrm{Var}_b(z \cdot v_k)$	as B1
B-rand (control)	as B1, but the s_k are applied to a random orthonormal basis, redrawn at every refresh	none	as B1

B-rand relaxes the same amount of SIGReg as B1, in directions unrelated to the shortcut score. If B-rand matches B1, the effect comes from weaker SIGReg, not from targeting.

6. Gate, shortcut score and budget definitions

Notation: $r = P(z_t, a_t) - \mathrm{sg}(z_\{t+1\})$ per sample b and latent dim j. $\varepsilon = 1e\text{-}6$. All gate and score statistics are computed under no_grad. $\bar{P}$ is an EMA copy of the predictor ($\tau_p = 0.99$) and $\bar{r} = \bar{P}(z_t, a_t) - \mathrm{sg}(z_\{t+1\})$ its residual on the same batch and the same current features.

Why lagged-predictor progress and not a loss slope. The loss on a direction can fall because the predictor learned it or because the encoder shrank that direction's variance. The second is exactly the deletion we want to detect, so a plain time-slope confounds the signal with the failure. Comparing two predictors on fixed current features isolates the predictor's learning. M1c keeps the naive slope to measure whether this confound matters.

M1a, per dimension. $LP_j = \mathrm{mean}_b(\bar{r}^2 - r^2)_j / (\mathrm{mean}_b\, \bar{r}^2_j + \varepsilon)$, smoothed with an EMA over steps. $g_j = 1 - \mathrm{clip}(LP_j / c, 0, 1)$, $c = 0.02$; $G = \mathrm{diag}(g)$. Weakness: SIGReg is rotation-invariant, so latent dims are not aligned with true factors.

M1b, random projections. Each step draw M/D independent random orthonormal bases of $\mathbb{R}^D$ (M = 4D directions, fresh every step). For each u: $LP_u = \mathrm{mean}_b[(\bar{r} \cdot u)^2 - (r \cdot u)^2] / (\mathrm{mean}_b(\bar{r} \cdot u)^2 + \varepsilon)$, $g_u = 1 - \mathrm{clip}(LP_u / c, 0, 1)$. $G = (D/M) \Sigma_u g_u u u^\top$, which is I exactly when all $g_u = 1$ and is equivariant to rotations of the latent.

M1c, naive loss slope. A fixed bank of M = 4D random directions drawn once. Per direction, fast and slow EMAs of mean_b(r·u)² (τ = 0.9, 0.99). s_u = (slow − fast) / (slow + ε); g_u = 1 − clip(s_u / c, 0, 1); G as in M1b. Pre-registered prediction: M1c protects whatever is currently being removed, noise included, and behaves close to A1.

M1d, residual eigenbasis (new). Same LP as M1b, measured in one basis U instead of fresh random bases. U is the eigenbasis of an EMA (τ = 0.99) of the residual covariance $r^T r$ / B, refreshed by eigh every 100 steps. Because U comes from past residuals, not the current batch, there is no selection bias. LP is within-batch, so no per-direction smoothing is needed and column identity matters only for logging. G = U diag(g) U^T. Note: do not use the eigenbasis of z itself; SIGReg pushes Cov(z) toward I, so its eigenvectors are nearly arbitrary.

Warm-up. At a loss plateau every LP variant reads ≈ 0, so all M1 variants use the C2 warm-up (G = 0 for 2k steps).

M2, ensemble disagreement. K = 5 heads with randomized prior networks (Osband et al. 2018) and Bernoulli(0.8) bootstrap masks. Epistemic e_bj = Var_k $\hat{z}^k$_bj; total u_bj = mean_k ($\hat{z}^k$ − z)²_bj. g_j = 1 − clip(β · mean_b e / mean_b u, 0, 1), G = diag(g). β calibrated once in Stage 0 so the gate is ≈ 1 on the direction best aligned with group 8 at A0 convergence.

M3, heteroscedastic. Predictor outputs μ and log v; the v-head is trained with Gaussian NLL against sg(μ) and sg(z_{t+1}). g_j = clip(mean_b v / mean_b r², 0, 1). Pre-registered prediction: fails, because v absorbs epistemic error.

Weak predictor and shortcut score (new, factor B only). The weak predictor p_w is linear in (sg(z_t), one-hot a_t), trained on its own MSE. V is the eigenbasis of an EMA of Cov(p_w), so its top directions are where the weak predictor explains the most variance. Per direction v_k keep EMAs (τ = 0.99) of var_k = Var_b(z_{t+1}·v_k) and err_k = mean_b((z_{t+1} − p_w)·v_k)². Then R^2_w,k = 1 − err_k / var_k and s_k = σ((R^2_w,k − τ_w) / T). Why V and not U: shortcut directions are nearly solved, so they sit in the low-eigenvalue tail of the residual covariance, where eigenvectors are poorly determined; V puts them at the top, where they are well conditioned.

Calibrating τ_w. Fixed once in Stage 0 from the Stage 0b R^2_lin table, then never retuned. Starting point τ_w = 0.8, T = 0.05. Pre-registered flagged set at D0: groups 1, 3, 4 and 7 (R^2_lin ⩾ 0.9); not flagged: 2, 5, 6, 8.

B1 / B2 / B-rand. B1: ω(w) = max(ω_min, 1 − Σ_k s_k (w·v_k)²) on each SIGReg slice, ω_min = 0.2 so SIGReg's anti-collapse role survives. B2 adds μ Σ_k s_k Var_b(z·v_k), μ = 0.01. B-rand: the same s_k on a random orthonormal basis, redrawn at every refresh.

Core training step (sketch, PyTorch).

```
x_t, a_t, x_n, gt = env.sample(B)                  # gt = 8 groups, for
probes only
z_t, z_n = enc(x_t), enc(x_n)
pred = P(z_t, a_t)
with torch.no_grad():
    r, r_lag = pred - z_n, P_lag(z_t, a_t) - z_n
    p_w = weak(z_t, a_t)
    stats.update(r, z_n, p_w)                      # EMA covs, U, V, R2_w, s
if arm == 'ema':
    tgt = enc_ema(x_n).detach()
else:
    with torch.no_grad():
        G = gate(arm, r, r_lag, stats, step)       # (D, D)
    zs = z_n.detach()
    tgt = zs + (z_n - zs) @ G                      # target-path gradient
mapped by G
z_all = torch.cat([z_t, z_n])
W = rand_unit_dirs(D, 256)
omega = slice_weights(W, stats.basis(bfac), stats.s, bfac)
loss = ((pred - tgt) ** 2).mean() + lam * (omega * ep_per_slice(z_all,
W)).mean()
if bfac == 'B2':
    loss = loss + mu * (stats.s * (z_all @ stats.V).var(0)).sum()
loss = loss + aux_losses(arm) + ((weak(z_t.detach(), a_t) - z_n.detach()) **
2).mean()
loss.backward(); opt.step(); update_emas(arm)

def lp_eig_gate(r, r_lag, U, c=0.02):              # M1d
    now, lag = (r @ U).pow(2).mean(0), (r_lag @ U).pow(2).mean(0)
    g = 1 - ((lag - now) / (lag + 1e-6) / c).clamp(0, 1)
    return (U * g) @ U.T

def slice_weights(W, basis, s, bfac, w_min=0.2):   # B1 / B2 / B-rand; basis
= V or random
    if bfac == 'B0':
        return torch.ones(W.shape[1])
    return (1 - ((W.T @ basis) ** 2 * s).sum(1)).clamp_min(w_min)
```

The M1b gate is unchanged from v0.3:

```
def lp_proj_gate(r, r_lag, D, M, c=0.02):          # M1b
    U = torch.cat([torch.linalg.qr(torch.randn(D, D))[0] for _ in range(M //
D)], 1)  # (D, M)
    now, lag = (r @ U).pow(2).mean(0), (r_lag @ U).pow(2).mean(0)
    g = 1 - ((lag - now) / (lag + 1e-6) / c).clamp(0, 1)
    return (D / M) * (U * g) @ U.T
```

7. Metrics

Metric	How	Read as
R^2_k, k = 1..8	Closed-form ridge from frozen z_t to each group, 8k fresh samples, every 500 steps	Retention profile over time
R^2_3 post-jump	Ridge R^2 for group 3 restricted to steps right after a jump	Are unpredictable events encoded at all
Trajectory shape	Max over training minus final R^2_k	Rise-then-fall = the race; never rises = allocation problem
Dynamics R^2_k	Ridge from $\hat{z}_{t+1}$ to group k at t+1, k = 4..7	Did the predictor learn the dynamics, not just the encoder keep the state
Variance share	Fraction of tr Cov(z) explained by each group (with degree-2 features)	Where the SIGReg budget goes
Effective dims per group (new)	Participation ratio of the ridge map from z to each group's degree-2 features	Above 2 for a slow group = redundant copies filling the budget (the Klindt Hermite effect)
Gate traces	Mean gate on the subspace best aligned with each group	Does the gate do what it claims
Shortcut-flag precision / recall (new)	s_k vs ground truth: flagged directions' alignment with groups 1, 3, 4, 7	Does the weak predictor flag what §6 pre-registered

Metric	How	Read as
Eigensubspace stability (new)	Principal angles between successive top-k subspaces of U and of V at each refresh	Whether per-direction states are trustworthy
Final MLP probe	2-layer MLP probe at end, held-out	Rules out "nonlinearly present"
Learning time t90	Stage 0b: first step where predictor-only $R^2_k \geq 0.9 \cdot R^2\max,k$	The learning-time axis, measured
R^2_lin per group (new)	Stage 0b: weak predictor on ground truth	Reference for the shortcut score; checks the §3 table

Key figures.

1. Race test: R^2_4 and R^2_5 over training per arm.
2. Calibration: final retention vs aleatoric fraction for groups 5, 6, 8 (ideal: high, high, zero).
3. Pareto: mean retention over groups 1–7 vs leakage of group 8, one point per arm with seed error bars. Stop-grad and EMA should move up and right; a successful gate moves up only.
4. Budget (new): variance share and effective dims per group for M1d, M1d+B1, M1d+B-rand at D = 16 and D = 48. A real allocation effect shows flagged groups losing surplus dims and groups 5–7 gaining them, at D = 16 only.

8. Protocol and budget

Stages 0–2 total 108 encoder runs plus 9 predictor-only runs, with up to 15 more conditional runs.

Stage	Content	Runs	Go / no-go
0. Reproduce the failure	A0 × λ {0.02, 0.1, 0.5} × w_fast {0.3, 1.0} × 2 seeds, D = 16, D0. Measure ρ per group. Calibrate β (M2) and τ_w, T (factor B)	12	Need A0 to drop group 5 ($R^2_5 < 0.3$) at some setting while keeping group 1 ($R^2_1 > 0.8$). Otherwise lower w_fast or raise difficulty first

Stage	Content	Runs	Go / no-go
0b. Learning time (no encoder)	Predictor and weak predictor on ground-truth state + action → next state, groups 1–7, D0/D1/D2 × 3 seeds. Per-group R^2 every 250 steps, t90	9 (seconds each)	Need t90(5) ⩾ 3× t90(4). If not at D0, try D1 as the default before changing the env
1. Main comparison	13 target arms (A0, A1 $\alpha \in \{0, 0.1\}$, A2 $\tau = 0.996$, M1a, M1b, M1c, M1d, M2, M3, C1, C2, C3) × 3 seeds, all at B0, D = 16, chosen failing setting	39	—
1-B. Budget factor (new)	A0+B1, M1d+B1, M1d+B2, M1d+B-rand × 3 seeds, D = 16	12	Read with the factor B table in §9
1b. Capacity	A0, A2, best M arm, A0+B1, M1d+B1 × 3 seeds at D = 48	15	Is the loss capacity competition or the target path? Does B only matter when the budget is tight?
1c. Middle capacity (conditional, new)	A0, M1d, M1d+B1 × 3 seeds at D = 24	9	Run only if B shows no effect at D = 16
2. Difficulty sweep	A0, A2, best of A1, best of M1b/M1d, M2 × D1, D2 × 3 seeds	30	Does the winner's margin hold as groups 5–7 get harder?
2-B (conditional, new)	Best G+B combination × D1, D2 × 3 seeds	6	Run only if B passed its §9 criterion
3. Optional	Pixels (§9 extensions)	—	Only if Stage 1 shows a clear winner

Stage 0b in detail. Same predictor architecture, optimizer, learning rate and batch size as the main runs, but the input is the true 2-dim values of groups 1–7 (cos/sin embedding for groups 4–7) plus the one-hot action, and the target is the true next-step values. No encoder, no SIGReg. The weak predictor is fitted in the same runs on the same inputs.

Checks:

1. t90(5) ⩾ 3× t90(4).
2. Group 6 plateaus at about 0.5 (0.45–0.55).
3. Group 7 sits between 4 and 5, or close to 5.
4. Group 3′s ceiling is about 0.90; group 8 stays near 0.
5. New: the weak predictor′s R^2 per group is within ±0.05 of R^2_lin in §3. If not, the §3 table or the weak predictor is wrong; fix before calibrating τ_w.
6. New: group 5 at D0 is per-action linear (§3). If check 1 fails at D0 but passes at D1, D1 becomes the Stage 1 default.

Caveat: with clean inputs these times are lower bounds, because in the real run the predictor also chases a moving latent. What matters is the ordering of the groups.

Speed tips. vmap or batch the 3 seeds of a cell into one process; probes are closed-form ridge; the two eigendecompositions are 16×16 (48×48 at most) every 100 steps, so their cost is negligible; log to a flat CSV per run.

9. Hypotheses, predicted outcomes, decision rules

Arm	Predicted retention (groups 1–7)	Predicted leakage (group 8)	If wrong, it means
A0	Keeps 1, 3, 4; drops 5, 6, 7 more than 4	Low	If A0 drops 4 as much as 5, learning time does not separate them. Read the A2 column before concluding anything
A1 / A2	Higher, but partial (Littwin bias)	Higher (no compression pressure)	If retention fully recovers, the target path is the whole story; adopt EMA / stop-grad and stop
M1a	Between A0 and M1b; seed-sensitive	Low	If equal to M1b, basis dependence does not matter here
M1b, M2	High, including 5 and 6	Low, close to A0	If no better than C1/C2, gating is unnecessary

Arm	Predicted retention (groups 1–7)	Predicted leakage (group 8)	If wrong, it means
M1c	High	High, like A1	If M1c ≈ M1b, the slope confound is negligible
M1d (new)	≈ M1b; at most slightly better	Low	At D = 16 random projections are barely diluted. If M1d clearly beats M1b, dilution matters and the eigenbasis is worth it at larger D
M3	Close to A0	Close to A0	If it works, residual-variance estimation is enough
C3	High	Low	Sanity bound; if low, capacity or probe problem
A0+B1 (new)	Partial gain on 5–7 at D = 16	Slightly higher	If it fully rescues 5, deletion is an allocation problem, not a target-path one
M1d+B1 / B2 (new)	Above M1d on 5–7 at D = 16; ≈ M1d at D = 48	≈ M1d	If equal to M1d at D = 16, there is no reallocatable budget; run Stage 1c
M1d+B-rand (new)	≈ M1d	≈ M1d	If ≈ M1d+B1, B works through weaker SIGReg, not targeting

Reading the 4 vs 5 test: both A0 and A2 are needed. Only the A2 (or A1) column can rule the target path in or out.

A0 (full gradient)	A2 (EMA / stop-grad)	Reading
drops 5, keeps 4	keeps both	Race confirmed. Only learning time differs, and removing the target path fixes it. Gates are worth building

A0 (full gradient)	A2 (EMA / stop-grad)	Reading
drops both	keeps both	The target path deletes even easy features. EMA / stop-grad already fix retention; gates matter only for the noise leakage they cause
drops both	drops both	Not the target path. Check timescale (5 vs 7), capacity (D = 48, factor B) and salience (w_fast)
any	keeps 4, drops 5	An extraction or learning problem, not deletion. Check Stage 0b t90 and the dynamics R^2 of group 5
keeps both	—	No failure on these groups. Stage 0 go/no-go not met; change the setting

Reading factor B (new). Compare at fixed G = M1d.

M1d+B1 vs M1d (D = 16)	M1d+B-rand vs M1d	M1d+B1 vs M1d (D = 48)	Reading
better by ⩾ 0.1 on R^2_5 or R^2_6	no better	no difference	Allocation effect confirmed: flagged directions hog the budget when it is tight. Keep B
better	equally better	—	The effect is weaker SIGReg, not targeting. Compare with the Stage 0 λ sweep; drop B
better	no better	also better	Not a capacity effect; B acts through another route. Investigate before keeping
no difference	—	—	No reallocatable budget at D = 16. Run Stage 1c (D = 24)
worse on groups 1–4 by > 0.1	—	—	Shrinking costs more than it frees. Drop B2 or lower μ

What A1/A2 do and do not show. They show what the encoder keeps when it is not rewarded for deleting. They do not show what is predictable in principle or how long each feature takes to learn: the encoder still decides what enters the latent through the

context path and SIGReg. The predictability ceiling comes from the environment design (R^2max), the learning time from Stage 0b.

Success criterion for a gate. Final R^2_5 and R^2_6 each at least 0.3 above A0 and at least 0.1 above the best of A1/A2/C1/C2; R^2_8 no more than 0.1 above A0; no group among 1–4 losing more than 0.1 versus A0; consistent across 3 seeds and holding at D1.

Success criterion for factor B (new). M1d+B beats M1d by at least 0.1 on R^2_5 or R^2_6 at D = 16 (or D = 24 if Stage 1c ran), M1d+B-rand does not, no group among 1–4 loses more than 0.1 versus M1d, R^2_8 no more than 0.1 above M1d, consistent across 3 seeds.

Extensions (deferred). (a) Pixels: slow groups as background texture or colour, fast groups as small dots on circles, group 8 as per-frame pixel noise (32×32), CNN encoder; the bridge to Craftax. (b) Task-relevant aleatoric features: attach a reward to group 3′s jumps. If group 3′s post-jump R^2 is low in every arm, that marks where a task signal (reward, IDM, BC) is still required.

10. Confounds and risks

- **Basis drift.** Per-dimension gates (M1a, M2, M3) assume a stable latent basis; M1b is basis-free; M1d tracks a data-defined basis. M1a vs M1b vs M1d measures how much this matters. Log eigensubspace stability; if principal angles between refreshes are large, slow the refresh or fall back to block-level gates.
- **Plateau ambiguity.** At a loss plateau, "learnable but not started" and "unlearnable" look identical to every signal here, including the weak predictor, which separates shortcut from semantic but not noise from not-yet-learned. This is the central scientific limit; warm-up and randomized priors are mitigations, and difficulty level D2 probes it directly.
- **Capacity.** At D = 16 there are only 2 spare dims, so factor B has almost nothing to reallocate. Stage 1b and the conditional Stage 1c separate this from the target-path effect.
- **SIGReg strength.** Too-strong λ collapses everything to a non-informative Gaussian (Klindt et al. grid); keep λ fixed across arms after Stage 0. B1 lowers effective strength on flagged slices; ω_min = 0.2 keeps SIGReg′s anti-collapse role, and B-rand controls for the strength change.
- **Weak predictor defines the shortcut (new).** Its capacity sets what gets flagged. R^2_lin shows group 7 (0.93 at D0) will be flagged although it is meant to be hard, and group 2 (0.49) will not. Calibrate τ_w once against the Stage 0b table and use the soft score, not a hard threshold.

- **Shortcut damping can cost real features (new).** Flagged groups 1, 3, 4, 7 must still be kept. B2′s shrink term is meant to remove surplus dims, not the features; the success criterion caps their loss at 0.1.
- **Probe choice.** Ridge can miss nonlinear encodings; the final MLP probe guards against declaring a feature lost when it is only rotated or warped.
- **Tuning asymmetry.** Gated arms and factor B add hyperparameters (c, β, τ_p, M, τ_w, T, ω_min, μ). Fix them in Stage 0 and do not retune per stage, or give baselines an equal tuning budget.

References

Carried over from v0.3; the two entries marked new were added from memory and not re-checked.

- Maes, Le Lidec, Scieur, LeCun, Balestriero. LeWorldModel: Stable End-to-End JEPA from Pixels. arXiv:2603.19312, 2026.
- Balestriero, LeCun. LeJEPA: Provable and Scalable SSL Without the Heuristics. arXiv:2511.08544, 2025.
- Klindt, LeCun, Balestriero. When Does LeJEPA Learn a World Model? arXiv:2605.26379, 2026.
- Sobal et al. Joint Embedding Predictive Architectures Focus on Slow Features. arXiv:2211.10831, 2022.
- Littwin et al. How JEPA Avoids Noisy Features: The Implicit Bias of Deep Linear Self Distillation Networks. NeurIPS 2024, arXiv:2407.03475.
- Qiao et al. AVL-JEPA: Preventing Causal Dynamics Information Collapse in JEPA World Models. arXiv:2610.03587, 2026.
- Mindermann et al. Prioritized Training on Points that are Learnable, Worth Learning, and Not Yet Learnt (RHO-LOSS). ICML 2022.
- Sekar et al. Planning to Explore via Self-Supervised World Models (Plan2Explore). ICML 2020.
- Lahlou et al. DEUP: Direct Epistemic Uncertainty Prediction. TMLR 2023.
- Osband, Aslanides, Cassirer. Randomized Prior Functions for Deep RL. NeurIPS 2018.
- Var-JEPA: A Variational Formulation of the Joint-Embedding Predictive Architecture. arXiv:2603.20111, 2026.
- Grill et al. BYOL, 2020; Chen & He. SimSiam, 2021; Assran et al. I-JEPA, 2023.

- New: Nam, Cha, Ahn, Lee, Shin. Learning from Failure: Training Debiased Classifier from Biased Classifier. NeurIPS 2020.
- New: Liu et al. Just Train Twice: Improving Group Robustness without Training Group Information. ICML 2021.